



Solana Private Channels Onchain

Security Assessment

Prepared for
Solana Foundation

Akash Gurugunti
Gabriel Ottoboni

Prepared by
Otter Audits, LLC

Prepared on
September 2nd, 2026

sudOu53r.ak@osec.io
ottoboni@osec.io

Table of Contents

1. Executive Summary	2
1.1. Overview	2
1.2. Key Findings	2
2. Scope	3
3. Findings	4
3.1. Findings summary	4
4. Suggestions	5
<u>OS-SPO-SUG-00</u> ● Unchecked Instance Version During Deserialization	6
<u>OS-SPO-SUG-01</u> ● Unchecked Token Account Data Size	7
<u>OS-SPO-SUG-02</u> ● SMT-Based Withdrawal Replay Tracking	9
<u>OS-SPO-SUG-03</u> ● Zero Escrow Instance Account Discriminator	10
<u>OS-SPO-SUG-04</u> ● Non-Anchor-Compatible Event CPI Encoding	11
<u>OS-SPO-SUG-05</u> ● Hardcoded Tree Height in Leaf Limit	12
<u>OS-SPO-SUG-06</u> ● Inconsistent Operator Account Naming	13
<u>OS-SPO-SUG-07</u> ● Redundant Signed Invocation in Deposit Transfer	14
<u>OS-SPO-SUG-08</u> ● Unnecessary Early Return in SMT Proof Verification	15
<u>OS-SPO-SUG-09</u> ● Duplicated Event Authority Validation	17
<u>OS-SPO-SUG-10</u> ● Duplicated SMT Proof Verification Logic	18
<u>OS-SPO-SUG-11</u> ● Unnecessary Accounts in Instruction Interfaces	19
A. Vulnerability Rating Scale	20
B. Procedure	21

1. Executive Summary

1.1. Overview

Solana Foundation engaged OtterSec to conduct a security assessment of the `escrow-program` and `withdraw-program` within Solana Private Channels. This assessment was conducted between June 25th and July 20th, 2026. For information on our auditing methodology, refer to [Appendix B](#).

1.2. Key Findings

We produced 12 findings throughout this audit engagement.

Among the most impactful issues, we identified that Instance deserialization does not reject a version mismatch, so a future layout change could silently decode old account bytes under the wrong schema and break instruction handling (● [OS-SPO-SUG-00](#)). We also found that the getter functions for retrieving mint decimals and token account balance rely on owner checks before unsafe unpacking, without first confirming the borrowed account data is large enough for the expected state layout, which weakens the safety boundary around token parsing (● [OS-SPO-SUG-01](#)).

Furthermore, we suggested an optimization as currently the withdrawal replay check uses a sparse Merkle tree root plus per-release sibling proofs to track consumed nonces. This protects against replay, but it makes every ReleaseFunds transfer carry and verify a full SMT proof, adding unnecessary compute and transaction overhead compared with a simpler bitmap-based design (● [OS-SPO-SUG-02](#)).

Another concern is that the InstanceDiscriminator is set to zero, rendering a valid escrow account prefix look indistinguishable from zero-filled account data and reducing robustness in account identification (● [OS-SPO-SUG-03](#)). In the event pipeline, the CPI format is not Anchor-compatible because the event authority seed and event discriminator encoding differ from the expected conventions, which can break downstream consumers and indexers that assume standard Anchor event metadata (● [OS-SPO-SUG-04](#)).

Lastly, we highlighted several instances of duplicate code (● [OS-SPO-SUG-09](#) and ● [OS-SPO-SUG-10](#)), and unnecessary accounts (● [OS-SPO-SUG-11](#)) that should be removed.

2. Scope

`escrow-program` is the mainnet-side custody program that locks deposited tokens in escrow, manages allowed mints and operators, and releases funds with SMT proof verification.

`withdraw-program` is the private-channel program that burns a user's channel tokens and emits a withdrawal event the indexer uses to trigger the mainnet release.

The source code was delivered to us in a Git repository at

<https://github.com/solana-foundation/solana-private-channels>. This audit was performed against commit [977327b](#).

3. Findings

Findings were rated using the criteria in [Appendix A](#).

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: **OS-SPO-ADV-*** for advisories and **OS-SPO-SUG-*** for suggestions.

3.1. Findings summary

Severity	Count	
Critical	0	
High	0	
Medium	0	
Low	0	
Informational	12	

4. Suggestions

Here, we present a discussion of the 12 suggestions we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-SPO-SUG-00	● Informational	○ Ack'd	Unchecked Instance Version During Deserialization
OS-SPO-SUG-01	● Informational	○ Ack'd	Unchecked Token Account Data Size
OS-SPO-SUG-02	● Informational	✓ Resolved	SMT-Based Withdrawal Replay Tracking
OS-SPO-SUG-03	● Informational	○ Ack'd	Zero Escrow Instance Account Discriminator
OS-SPO-SUG-04	● Informational	○ Ack'd	Non-Anchor-Compatible Event CPI Encoding
OS-SPO-SUG-05	● Informational	○ Ack'd	Hardcoded Tree Height in Leaf Limit
OS-SPO-SUG-06	● Informational	○ Ack'd	Inconsistent Operator Account Naming
OS-SPO-SUG-07	● Informational	○ Ack'd	Redundant Signed Invocation in Deposit Transfer
OS-SPO-SUG-08	● Informational	○ Ack'd	Unnecessary Early Return in SMT Proof Verification
OS-SPO-SUG-09	● Informational	○ Ack'd	Duplicated Event Authority Validation
OS-SPO-SUG-10	● Informational	○ Ack'd	Duplicated SMT Proof Verification Logic
OS-SPO-SUG-11	● Informational	○ Ack'd	Unnecessary Accounts in Instruction Interfaces

Unchecked Instance Version During Deserialization

ID	Severity	Status
OS-SPO-SUG-00	● Informational	○ Ack'd

Description

The escrow program stores a version byte in each `Instance` account when the account is created. The initialization path constructs the instance with the current `INSTANCE_VERSION`, and the account layout documented for `Instance` includes `version` as a dedicated field.

`Instance::try_from_bytes` reads the stored version and returns it as part of the deserialized account, but the loader only validates the account discriminator before parsing the remaining fields. The escrow instruction processors that consume instance state continue with their instruction-specific checks without asserting that the parsed version matches the current layout version.

As a result, if a future `v2` changes the `Instance` layout, existing deserialization could silently interpret account bytes under the wrong schema unless version validation is added before that release.

Remediation

Update `Instance::try_from_bytes` to reject accounts whose parsed `version` does not equal the program's supported `INSTANCE_VERSION`, returning an explicit account-data error before any instruction uses the decoded fields.

Patch

Solana Foundation acknowledged this recommendation.

Unchecked Token Account Data Size

ID	Severity	Status
OS-SPO-SUG-01	● Informational	○ Ack'd

Description

`get_mint_decimals` selects the parsing path only from the account owner, accepting accounts owned by either supported token program before constructing an unchecked mint state view. Without first confirming that `mint_info` has the minimum mint state size for that token program, an owner-matching account that is not actually a mint can reach `TokenMint::from_bytes_unchecked`. This is not expected to cause undefined behavior in this case because token accounts are larger than mint accounts, but it still weakens the safety contract implied by using an unchecked state conversion.

```
private-channel-escrow-program/program/src/processor/shared/token_utils.rs
```

```

99  pub fn get_mint_decimals(mint_info: &AccountView) -> Result<u8, ProgramError> {
100      if mint_info.owned_by(&TOKEN_PROGRAM_ID) {
101          let data = mint_info.try_borrow()?;
102          let mint = unsafe { TokenMint::from_bytes_unchecked(&data) };
103          return Ok(mint.decimals());
104      }
105      if mint_info.owned_by(&TOKEN_2022_PROGRAM_ID) {
106          let data = mint_info.try_borrow()?;
107          let mint = unsafe { Token2022Mint::from_bytes_unchecked(&data) };
108          return Ok(mint.decimals());
109      }
110      Err(PrivateChannelEscrowProgramError::InvalidMint.into())
111  }
```

`get_mint_decimals` uses owner checks before unchecked mint parsing.

`get_token_account_balance` follows the same pattern for token account balances. Ownership by the legacy or Token-2022 program is treated as sufficient before constructing an unchecked token-account state view. The helper is used by deposit and release processing to measure escrow balance changes around token transfers, so rejecting incorrectly sized or non-token-account data at this boundary would make the helper's assumptions explicit before those balance calculations depend on them.

```
private-channel-escrow-program/program/src/processor/shared/token_utils.rs
```

```

84  pub fn get_token_account_balance(info: &AccountView) -> Result<u64, ProgramError> {
85      if info.owned_by(&TOKEN_PROGRAM_ID) {
86          let data = info.try_borrow()?;
87          let account = unsafe { TokenAccount::from_bytes_unchecked(&data) };
88          return Ok(account.amount());
89      }
```

```

90     if info.owned_by(&TOKEN_2022_PROGRAM_ID) {
91         let data = info.try_borrow()?;
92         let account = unsafe { Token2022Account::from_bytes_unchecked(&data) };
93         return Ok(account.amount());
94     }
95     Err(PrivateChannelEscrowProgramError::InvalidTokenAccount.into())
96 }

```

get_token_account_balance uses owner checks before unchecked token-account parsing.

Remediation

Before each unchecked token state conversion, validate the borrowed account data against the expected state size for the specific token program and state type.

Patch

Solana Foundation acknowledged this recommendation.

SMT-Based Withdrawal Replay Tracking

ID	Severity	Status
OS-SPO-SUG-02	● Informational	✓ Resolved

Description

The escrow program currently uses a sparse Merkle tree root to track consumed withdrawal nonces for `ReleaseFunds`. The on-chain `Instance` account stores `withdrawal_transactions_root` and `current_tree_index`, and `ReleaseFunds` validates that the submitted `transaction_nonce` belongs to the active tree before verifying both an SMT exclusion proof against the current root and an SMT inclusion proof against the proposed new root. After the token transfer, the processor writes the supplied new root back into the instance state. This design provides replay protection, but it requires every release to carry and verify a 16-level sibling proof.

The off-chain operator mirrors this structure. It reconstructs the local SMT from completed withdrawal nonces for the current tree window, checks the computed root against the on-chain instance root, generates an exclusion proof immediately before sending a release, inserts the nonce locally to derive the new root, flattens the proof into `sibling_proofs`, and includes that proof plus the new root in the `ReleaseFunds` instruction. Because the tree only records whether a nonce in the current nonce window has already been consumed, a fixed on-chain nonce bitmap with the same nonce-to-window mapping and reset rotation performs the same replay-protection role with less per-release work.

Remediation

Replace the withdrawal SMT with an 8 KB bitmap per instance, which would avoid transmitting sibling proofs in each `ReleaseFunds` transaction, remove the on-chain hash-proof verification path, and simplify the operator-side SMT maintenance and proof-regeneration logic. It cuts per-release compute by ~1000x, trimming 544 bytes per `ReleaseFunds` transaction, deleting significant on-chain and off-chain code, and holding rent at a fixed 0.058 SOL/instance.

Patch

Resolved in [PR#244](#).

Zero Escrow Instance Account Discriminator

ID	Severity	Status
OS-SPO-SUG-03	● Informational	○ Ack'd

Description

The escrow program defines one-byte account discriminators for its serialized account types, and `PrivateChannelEscrowAccountDiscriminators` assigns `InstanceDiscriminator` the value `0`, as shown in the account discriminator enumeration. Because newly allocated or otherwise zero-filled account data also begins with zero bytes, using `0` for a valid account type makes the serialized `Instance` account prefix less visually and programmatically distinct from empty account data.

```
private-channel-escrow-program/program/src/state/discriminator.rs
```

```

5  pub trait Discriminator {
6      const DISCRIMINATOR: u8;
7  }
8
9  #[repr(u8)]
10 pub enum PrivateChannelEscrowAccountDiscriminators {
11     InstanceDiscriminator = 0,
12     OperatorDiscriminator = 1,
13     AllowedMintDiscriminator = 2,
14 }
```

Remediation

Assign `InstanceDiscriminator` a non-zero value and keep all escrow account discriminator values unique.

Patch

Solana Foundation acknowledged this recommendation.

Non-Anchor-Compatible Event CPI Encoding

ID	Severity	Status
OS-SPO-SUG-04	● Informational	○ Ack'd

Description

For the event CPI to be fully consistent with Anchor conventions, the event authority seed and event discriminator format should match Anchor's expected values. Set `EVENT_AUTHORITY_SEED` to `__event_authority`, and use the first eight bytes of `sha256("event:<EventStruct>")` as each event discriminator instead of a single-byte `EventDiscriminators` value.

Remediation

Align the event CPI format with Anchor conventions by setting `EVENT_AUTHORITY_SEED` to `__event_authority` and using `sha256("event:<EventStruct>")[..8]` event discriminators instead of single-byte `EventDiscriminators`.

Patch

Solana Foundation acknowledged this recommendation.

Hardcoded Tree Height in Leaf Limit

ID	Severity	Status
OS-SPO-SUG-05	● Informational	○ Ack'd

Description

The current implementation defines a `TREE_HEIGHT` constant and then calculates `MAX_TREE_LEAVES` with a separate numeric exponent, as shown in below. Because `MAX_TREE_LEAVES` represents the number of leaves implied by the tree height, keeping the exponent as a literal duplicates the same configuration value in two places.

```
indexer/src/operator/constants.rs
```

```
1 use crate::operator::{const_hash_combine, EMPTY_LEAF};
2 pub const TREE_HEIGHT: usize = 16;
3 pub const MAX_TREE_LEAVES: usize = 2_usize.pow(16); // 65,536 leaves in production
```

This makes the SMT configuration easier to misconfigure during future maintenance. If `TREE_HEIGHT` is changed without updating the exponent used by `MAX_TREE_LEAVES`, downstream operator logic may become inconsistent with the configured tree height.

Remediation

Derive `MAX_TREE_LEAVES` directly from `TREE_HEIGHT`.

Patch

Solana Foundation acknowledged this recommendation.

Inconsistent Operator Account Naming

ID	Severity	Status
OS-SPO-SUG-06	● Informational	○ Ack'd

Description

There is naming-consistency issue in the escrow program where `operator_pda` should be renamed to `operator_info` so it matches the naming convention used across the program's instructions.

Remediation

Rename `operator_pda` to `operator_info`, to be consistent with the naming convention being used in all the instructions in the program.

Patch

Solana Foundation acknowledged this recommendation.

Redundant Signed Invocation in Deposit Transfer

ID	Severity	Status
OS-SPO-SUG-07	● Informational	○ Ack'd

Description

`process_deposit` transfers tokens from the depositor's associated token account into the instance escrow associated token account after validating the user signer, mint, allowed mint PDA, and both token accounts. The transfer authority for this CPI is `user_info`, which is already required to sign the deposit instruction, so the transfer does not require any program-derived address signer seeds. Replace the `invoke_signed` call in `process_deposit` with `invoke`.

```
private-channel-escrow-program/program/src/processor/deposit.rs
1  pub fn process_deposit(...) -> ProgramResult {
2      [...]
3      TransferChecked2022 {
4          from: user_ata_info,
5          to: instance_ata_info,
6          authority: user_info,
7          amount: args.amount,
8          token_program: token_program_info.address(),
9          mint: mint_info,
10         decimals: get_mint_decimals(mint_info)?,
11     }
12     .invoke_signed(&[])?;
13     [...]
14 }
```

Deposit transfer uses signed invocation without PDA signer seeds

Because the signed CPI is invoked with an empty signer seed slice, it appears functionally equivalent to a plain invocation in this context. This does not indicate an authorization bypass or loss of funds based on the surrounding control flow; it is an unnecessary use of the signed invocation form that can make the authority model less clear to readers and maintainers.

The same escrow program uses signed invocation with explicit instance PDA seeds in the release flow, where the instance account is the transfer authority. That contrast supports using the simpler unsigned CPI form for deposits, where the user remains the token authority.

Remediation

Replace the empty `invoke_signed` call in `process_deposit` with the plain CPI invocation method for the `TransferChecked2022` transfer. Keep the existing `user_info` signer requirement and ATA validation unchanged, since those checks establish the depositor as the token transfer authority.

Patch

Solana Foundation acknowledged this recommendation.

Unnecessary Early Return in SMT Proof Verification

ID	Severity	Status
OS-SPO-SUG-08	● Informational	○ Ack'd

Description

`verify_smt_exclusion_proof` computes a Merkle path from an empty leaf using the nonce-derived leaf position and the fixed-size sibling proof array. The function already validates the computed root after processing the proof, but it also accepts the proof immediately if an intermediate hash equals the current root, as shown in the exclusion verifier.

```
private-channel-escrow-program/program/src/processor/shared/smt_utils.rs
```

```

1  pub fn verify_smt_exclusion_proof(
2      current_root: &[u8; 32],
3      transaction_nonce: u64,
4      sibling_proofs: &[[u8; 32]; TREE_HEIGHT],
5  ) -> ProgramResult {
6      // Use nonce bits directly for path determination
7      let leaf_position = transaction_nonce as usize % MAX_TREE_LEAVES;
8
9      // Start with empty leaf (null hash for exclusion proof)
10     let mut current_hash = EMPTY_LEAF;
11
12     // Traverse from leaf to root using sibling proofs
13     [...]
14
15     // Final check: computed root must match current root for valid exclusion
16     if current_hash != *current_root {
17         return Err(PrivateChannelEscrowProgramError::InvalidSmtProof.into());
18     }
19
20     Ok(())
21 }
```

`verify_smt_inclusion_proof` follows the same structure for the proposed new withdrawal root, starting from the non-empty leaf hash and traversing the supplied sibling proofs. It likewise returns success when an intermediate hash equals the target root before the full proof has been consumed, as shown in the inclusion verifier.

```
private-channel-escrow-program/program/src/processor/shared/smt_utils.rs
```

```

1  pub fn verify_smt_inclusion_proof(
2      new_root: &[u8; 32],
3      transaction_nonce: u64,
4      sibling_proofs: &[[u8; 32]; TREE_HEIGHT],
```

```

5  ) -> ProgramResult {
6      // Use nonce bits directly for path determination
7      let leaf_position = transaction_nonce as usize % MAX_TREE_LEAVES;
8
9      // Start with non-empty leaf (what's actually stored in the SMT for inclusion proof)
10     let mut current_hash = NON_EMPTY_LEAF_HASH;
11
12     // Traverse from leaf to root using sibling proofs
13     [...]
14
15     // Final check: computed root must match new root for valid inclusion
16     if current_hash != *new_root {
17         return Err(PrivateChannelEscrowProgramError::InvalidSmtProof.into());
18     }
19
20     Ok(())
21 }

```

The early-return branches are unnecessary for the documented fixed-height SMT verification flow and this intermediate-root case is not intended to occur.

Remediation

Remove the intermediate `current_hash == root` success checks from both SMT proof verification loops and rely on the existing final root comparison after all `TREE_HEIGHT` siblings have been processed.

Patch

Solana Foundation acknowledged this recommendation.

Duplicated Event Authority Validation

ID	Severity	Status
OS-SPO-SUG-09	● Informational	○ Ack'd

Description

`process_emit_event` performs its own event authority PDA comparison before checking the account signer status. This validation should use the predefined `validate_event_authority!` macro instead of the current inline condition, because keeping the PDA check directly in the `EmitEvent` processor duplicates validation logic that already exists in shared code.

```
private-channel-escrow-program/program/src/processor/process_emit_event.rs
```

```

9  pub fn process_emit_event(_program_id: &Address, accounts: &[AccountView]) ->
    ProgramResult {
10      let [event_authority] = accounts else {
11        return Err(ProgramError::NotEnoughAccountKeys);
12      };
13      if event_authority.address().ne(&event_authority_pda::ID) {
14        return Err(PrivateChannelEscrowProgramError::InvalidEventAuthority.into());
15      }
16      verify_signer(event_authority, false)?;
17      Ok(())
18    }

```

Remediation

Use the existing `validate_event_authority!` macro in `process_emit_event` for the event authority PDA check.

Patch

Solana Foundation acknowledged this recommendation.

Duplicated SMT Proof Verification Logic

ID	Severity	Status
OS-SPO-SUG-10	● Informational	○ Ack'd

Description

The SMT utility implements separate exclusion and inclusion proof verifiers in `SparseMerkleTreeUtils`. In `verify_smt_exclusion_proof`, the function derives the leaf position from the transaction nonce, starts traversal from `EMPTY_LEAF`, walks the sibling proof path with order-sensitive hash combination, accepts an early root match, and otherwise returns an invalid proof error if the computed root does not match.

The corresponding `verify_smt_inclusion_proof` function repeats the same path derivation, sibling traversal, early termination, final root comparison, and invalid-proof error handling, but initializes traversal from `NON_EMPTY_LEAF_HASH` and compares against `new_root`. The duplicated control flow increases maintenance burden because changes to nonce path calculation, sibling hashing, early termination, or error handling must be kept synchronized across both functions.

Remediation

Extract the shared SMT path traversal into a private helper that can be used by both `verify_smt_exclusion_proof` and `verify_smt_inclusion_proof`.

Patch

Solana Foundation acknowledged this recommendation.

Unnecessary Accounts in Instruction Interfaces

ID	Severity	Status
OS-SPO-SUG-11	● Informational	○ Ack'd

Description

Several instruction interfaces include accounts that are unnecessary for the corresponding operation. The escrow program's `deposit` instruction does not need `system_program`, `associated_token_program`, or payer `release_funds` does not need `associated_token_program` or payer `block_mint` and `remove_operator` do not need `system_program` and `set_new_admin` and `reset_smt_root` do not need payer. The withdraw program's `withdraw_funds` instruction similarly does not need `associated_token_program`.

Remediation

Remove the unnecessary accounts listed above.

Patch

Solana Foundation acknowledged this recommendation.

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
● Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
● High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
● Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
● Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
● Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of  Resolved,  Ack'd, or  To-do.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process